it is 0.5. The parameter *random_state* determines the random number generation for creating the data set. The method *make_blob( )* returns the generated samples (stored in array *X*, in this case) and the integer labels for cluster membership of each sample (stored in *y*, in this case). Finally, lines 6 and 7 plot and display the sample data generated in a two-dimensional plane. Figure 3 shows this plot of the generated sample data.

Now that we have the necessary data, let us try to identify clusters from this data. The following lines of code do exactly that.

```
8.  kmeans = KMeans(n_clusters=2,
    init='k-means++', max_iter=1000,
    random_state=0)
9.  predict_y = kmeans.fit_predict(X)
10. plt.scatter(X[:,0], X[:,1])
11. plt.scatter(kmeans.cluster_centers_
    [:, 0], kmeans.cluster_centers_
    [:, 1], s=500, c='green')
12. plt.show( )
```

Now, let us try to understand the above lines of code. Line 8 calls the class *KMeans* to create an object of it called *kmeans*. Let us discuss the parameters passed to this class. The parameter *n_clusters* fixes the number of clusters to be formed as well as the number of cluster centres to be generated. In our case it is 2. The parameter *init* decides the method for initialisation. The two possible options for the parameter *init* are *'k-means++'* and *'random'*. This parameter decides how cluster centres are identified. We chose *'k-means++'* because it results in faster convergence while finding the cluster centres. The parameter *max_iter* decides the maximum number of iterations of the K-Means algorithm on a single execution. In this case, it is 1000. The parameter *random_state* determines the random number generated for initialising the cluster centres. In this case, it is 0. Line 9 uses the method *fit_predict( )* of the class *KMeans()* to compute the cluster centres and also predicts the cluster to which each sample in the sample data belongs. Line 10 again plots the sample data generated. Line 11 plots the cluster centres in green colour. Finally, Line 12 displays the images plotted. Figure 4 shows this plot of the sample data generated and the cluster centres identified. Thus we have implemented our first, though simple, unsupervised learning algorithm.

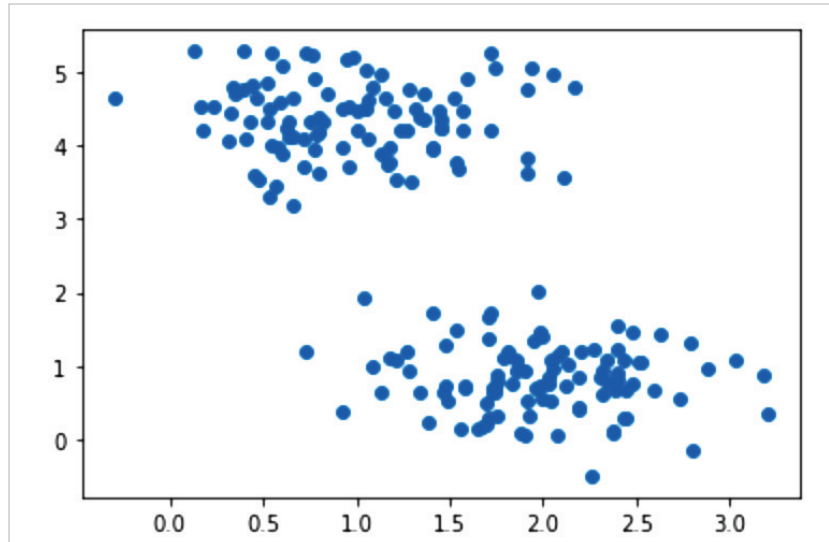

Figure 3: Sample data generated

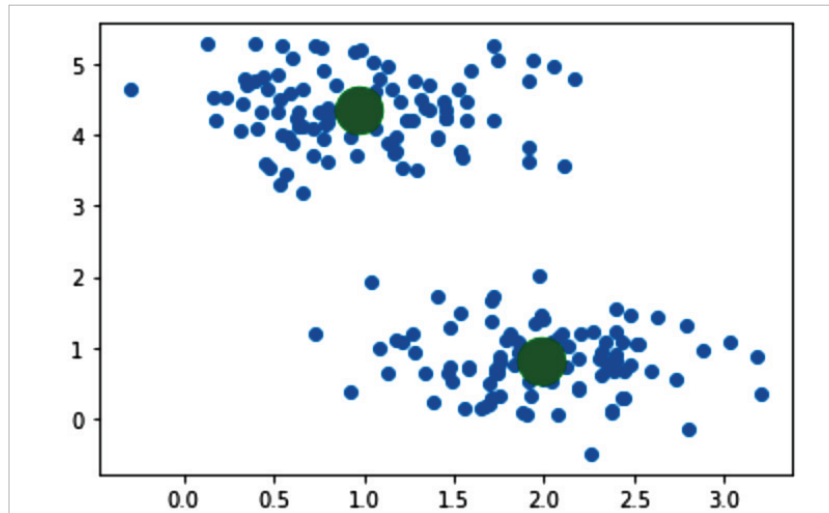

Figure 4: Cluster centres identified by K-Means clustering

An introduction to PyTorch

Now let us make ourselves familiar with PyTorch, a powerful machine learning framework based on the Torch library. Torch is an open source machine learning library and a scientific computing framework based on a programming language called Lua. PyTorch is free and open source software released under the modified BSD licence. PyTorch can work in both CPU mode and GPU mode. But in order to work with the GPU mode of PyTorch, we need to have an Nvidia graphics card and